Code Standards

Condensed Developer Guide

1. Naming Conventions

- **Variables & Functions**: Use **camelCase** with descriptive names. Boolean variables start with *is*, *has*, or *can*. Functions should be verbs (e.g., fetchUserById).
- **Classes & Types**: Use **PascalCase**. Classes are nouns (UserService), interfaces may be prefixed with *I*, enums use PascalCase with members in UPPER_SNAKE_CASE.
- **Constants**: Use **UPPER_SNAKE_CASE** for global values (SECONDS_PER_DAY). Avoid magic numbers.
- **Files & Folders**: Use **kebab-case** for portability. Group by feature, not type.

2. Code Formatting

- **Indentation & Spacing**: Consistent spaces (2 or 4). Use .editorconfig or Prettier. Keep line length within 80–120 characters.
- **Braces & Brackets**: Follow K&R style, always use braces even for single-line blocks.
- **Linters & Formatters**: Automate style enforcement (ESLint, Prettier, Black). Run formatters in pre-commit hooks.

3. Commenting & Documentation

- **Self-Documenting Code**: Write clear code; comments explain *why*, not *what*.
- **Docstrings/JSDoc**: Public APIs must document parameters, return types, and exceptions.
- **TODO/FIXME**: Use standardized tags for notes, link to tickets when possible.

4. Error Handling

- **Never Swallow Errors**: Always log or re-throw with context.
- **Custom Error Classes**: Extend base error types, add status codes for clarity.
- **Fail Fast**: Validate inputs early using schema libraries (Zod, Joi, Pydantic).

5. Code Structure & Organisation

- **DRY Principle**: Avoid duplication; extract shared logic into utilities.
- **Function Size**: Keep functions small, focused, ideally under 20–30 lines.
- **Separation of Concerns**: Controllers handle HTTP, services handle logic, repositories handle data.

6. Version Control Standards

- **Conventional Commits**: Use structured commit messages (feat(auth): add JWT rotation).
- **Branch Naming**: Use lowercase, hyphen-separated names linked to tickets (feature/AUTH-42-oauth-login).

- **Pull Requests**: Keep PRs small, focused, and reviewed before merging. Include what changed, why, and how to test.

7. Testing Standards

- **Test Naming**: Write descriptive names that read like specifications (returns 10% off for premium users).
- **AAA Pattern**: Arrange, Act, Assert for clarity.
- **Coverage**: Focus on critical paths, edge cases, and error handling. Aim for ~80% meaningful coverage.